ECE 572 Assignment 2 Report

Course: ECE 572; Summer 2026 **Instructor**: Dr. Ardeshir Shojaeinasab
Student Name: Brandon Rogers
Student ID: V00759375
Assignment: Assignment 2
Date: July 11, 2026
GitHub Repository: https://github.com/uvicrepo/ECE572_SecureText_brandon

---

Executive Summary

This assignment builds on the SecureText messenger from Assignment 1 by adding three layers of authentication: TOTP based two factor authentication, GitHub OAuth login, and a set of Zero Trust features on top of both. The goal across all three tasks was to move authentication away from a single password check toward something that keeps verifying identity throughout a session rather than trusting it once at login.

TOTP was added using pyotp and PyCryptodome, following RFC 6238 for the algorithm parameters and encrypting secrets at rest with AES-256-GCM. GitHub OAuth was added as a second login path using the standard authorization code flow, adapted for a console app by having the user paste back the redirect URL instead of running a local callback server. The Zero Trust task added a basic HMAC challenge-response mechanism, role based access control with user and admin roles, session timeouts based on action count, TOTP gated password resets, and logging with a basic alerting mechanism for repeated failed logins. Several pre-existing bugs from Assignment 1 were also found and fixed along the way, since they blocked testing of the new features.

Table of Contents

1. Introduction
2. Task Implementation
   - Task 4
   - Task 5
   - Task 6
3. Security Analysis
4. Attack Demonstrations
5. Performance Evaluation
6. Lessons Learned
7. Conclusion
8. References

---

1. Introduction

1.1 Objective

The objective of this assignment was to add stronger authentication to SecureText without changing the core messaging functionality built in Assignment 1. This meant adding a second authentication factor with TOTP, adding support for logging in with an existing GitHub account instead of a local password, and applying Zero Trust ideas such as continuous verification, least privilege access, and session expiration on top of both.

1.2 Scope

All three tasks were implemented in full except for PKCE on the OAuth flow, which was left out since it was listed as optional bonus work and the time available did not justify it for the points on offer. Each task was built as its own deliverable file, starting from the previous task's file, so the final Zero Trust version contains all of the work from Tasks 4, 5, and 6 together.

1.3 Environment Setup

- **Operating System**: Linux
- **Python Version**: 3.12
- **Key Libraries Used**: pyotp, qrcode, pycryptodome, requests, plus the standard library hmac, hashlib, and secrets modules
- **Development Tools**: VS Code with Pylance, git and GitHub for version control, Claude LLM for report summary (based off self-authored rough copy developed by following local readme.md) and troubleshooting as noted in the AIGEN.md file.

---

2. Task Implementation

2.1 Task 4: TOTP Two Factor Authentication

2.1.2 Implementation Details Add a second authentication factor so that a stolen or guessed password is not enough on its own to log in. This also meant fixing the way passwords were stored and hashed so the whole login flow held together end to end.

Key Components: - Component 1: TOTP secret generation with pyotp's random_base32(), which by default produces a 20 byte secret. This happens to already match the RFC 6238 recommendation that a key be as long as the HMAC output it will be used with, in this case HMAC-SHA1, so no custom key length logic was needed beyond confirming the default was already correct - Component

2: Secrets encrypted at rest with AES-256-GCM through PyCryptodome, stored as one hex string containing the nonce, authentication tag, and ciphertext - Component 3: QR code provisioning using the provisioning_uri() method from pyotp combined with the qrcode library's ASCII output, so a phone authenticator app can scan the secret in during account creation - Component 4: Login now checks TOTP with a plus or minus one time step tolerance for clock drift, and locks out further TOTP attempts for a short period after repeated failures

Code Snippet (Key Implementation):

```
def encrypt_totp_secret(secret):
    cipher = AES.new(TOTP_KEY, AES.MODE_GCM)
    ciphertext, tag = cipher.encrypt_and_digest(secret.encode())
    return (bytes(cipher.nonce) + bytes(tag) + bytes(ciphertext)).hex()


def verify_totp(secret, totp_input):
    totp = pyotp.TOTP(secret, digest = TOTP_DIGEST)
    if totp.verify(totp_input, valid_window=1):  # Allow for a 1-step window to account for
        return True
    else:
        time.sleep(10)
        print(f"[TOTP] VERIFICATION FAILED - Invalid TOTP code")
        return False
```

2.1.3 Challenges and Solutions The biggest early issue was that TOTP secrets need to be recoverable, not just verifiable, since the server has to compute the current expected code from the stored secret at login time. That ruled out hashing them and led to reversible AES-GCM encryption instead, implemented directly with PyCryptodome rather than a higher level wrapper.

I also found a login bug while wiring TOTP in, where the password check in authenticate() was inverted so that a correct password caused login to fail. This was not related to TOTP but blocked all testing until it was fixed. Separately, the original login flow had the client compute its own "actual" TOTP code and send it to the server alongside the user's entered code, which completely defeats the point of a second factor since the server should be deriving the expected code from the secret it already holds, not trusting a value from the client.

2.1.4 Testing and Validation Login was tested with a valid TOTP code generated from the stored secret through a quick pyotp one-liner, confirming successful login. Testing with a wrong code confirmed a rejection plus an intentional delay. Repeated wrong TOTP attempts triggered the lockout, and codes were accepted slightly outside the exact 30 second window to confirm the clock skew tolerance was working.

Test Cases Evidence:

```
1. Create Account
2. Login
3. Reset Password
4. Exit
Choose an option: 1

=== Create Account ===
Enter username: Emma
Enter password: emma123
Account created successfully
[TOTP] Scan this QR code with your authenticator app (or en
ter the secret manually: SVEOKS7XFIS2BAU6AX4BEITM5OVY3WMV)
```





2.2 Task 5: OAuth Integration

2.2.1 Objective Allow login through an existing GitHub account as an alternative to the local username and password flow, and use that as the basis for granting admin access later in Task 6.

2.2.2 Implementation Details The OAuth flow follows the standard authorization code grant, adapted for a console application. The client opens the authorization URL in the default browser, the user logs in and approves the app on GitHub, and is redirected to a localhost address that nothing is actually listening on. Since the code and state parameters are visible in that

URL regardless of whether anything is listening, the user copies the full URL and pastes it back into the console, and the client sends it to the server for parsing.

A random state value is generated per connection before the authorization URL is built, and is checked against the value returned in the pasted URL before anything else happens, which is what prevents a CSRF style attack against this flow. The exchange of the authorization code for an access token happens entirely on the server side using the GitHub client secret, which is read from an environment variable and never sent to the client. The access token itself is only ever held in memory for the duration of that one exchange and is never written to disk.

Account linking checks the primary email returned by GitHub's /user/emails endpoint against the email stored on local accounts. If it matches, the GitHub login is treated as that existing account. If the GitHub username collides with a different local account, the new account is suffixed with _github and the user is told this happened in the login message rather than it happening silently.

Key Components: - Component 1: OAUTH_START and OAUTH_CALLBACK commands handling the two round trips of the flow - Component 2: CSRF protection through a per connection random state value - Component 3: Server side only token exchange, with the client secret kept out of source control through an environment variable - Component 4: Email based account linking with a visible message when a username conflict occurs

Code Snippet (Key Implementation):

```
    def build_github_authorize_url(self, state):
        params = {
            'client_id': GITHUB_CLIENT_ID,
            'redirect_uri': GITHUB_REDIRECT_URI,
            'scope': 'read:user user:email',
            'state': state
        }
        return GITHUB_AUTHORIZE_URL + '?' + urllib.parse.urlencode(params)

    def oauth_login(self, conn, redirect_url):
        expected_state = self.oauth_pending_state.pop(conn, None)
        parsed = urllib.parse.urlparse(redirect_url)
        qs = urllib.parse.parse_qs(parsed.query)
        code = qs.get('code', [None])[0]
        returned_state = qs.get('state', [None])[0]

        if not expected_state or returned_state != expected_state:
            return False, "OAuth state mismatch - login aborted", None
        if not code:
            return False, "No authorization code found in that URL", None
```

```
token_resp = requests.post(GITHUB_TOKEN_URL, headers={'Accept': 'application/json'},
    'client_id': GITHUB_CLIENT_ID,
    'client_secret': GITHUB_CLIENT_SECRET,
    'code': code,
    'redirect_uri': GITHUB_REDIRECT_URI
})
access_token = token_resp.json().get('access_token')
if not access_token:
    return False, "GitHub did not return an access token", None

auth_header = {'Authorization': f'Bearer {access_token}'}
github_user = requests.get(GITHUB_USER_URL, headers=auth_header).json()
github_username = github_user.get('login')

emails = requests.get(GITHUB_EMAILS_URL, headers=auth_header).json()
primary_email = next((e['email'] for e in emails if e.get('primary')), None)

matched_username = None
if primary_email:
    for uname, record in self.users.items():
        if record.get('email') == primary_email:
            matched_username = uname
            break

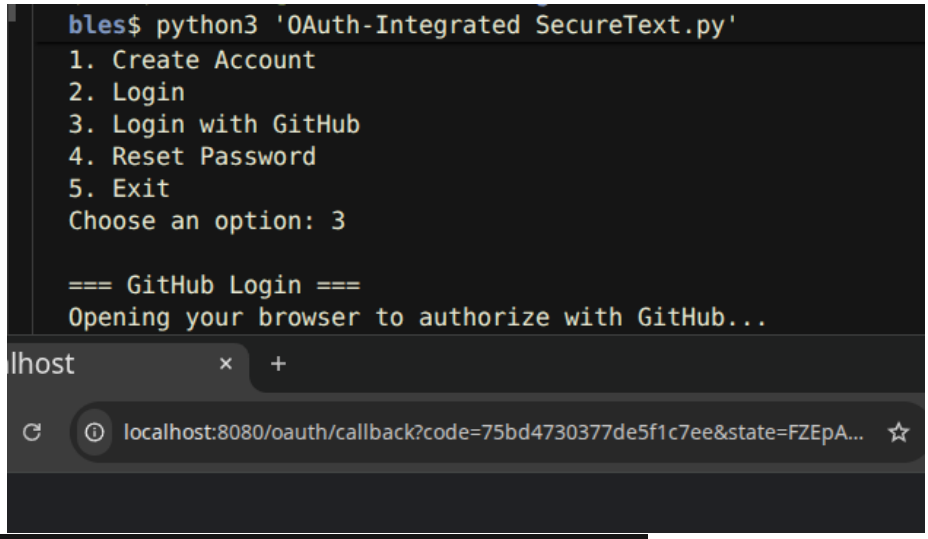
if matched_username:
    login_username = matched_username
    conflict_note = ""
else:
    login_username = github_username
    conflict_note = ""
    if login_username in self.users:
        login_username = f"{github_username}_github"
        conflict_note = f" (username '{github_username}' already taken; defaulting t
    if login_username not in self.users:
        self.users[login_username] = {
            'email': primary_email,
            'oauth_provider': 'github',
            'created_at': datetime.now().isoformat()
        }
        self.save_users()

return True, f"Logged in via GitHub as {login_username}{conflict_note}", login_usern
```

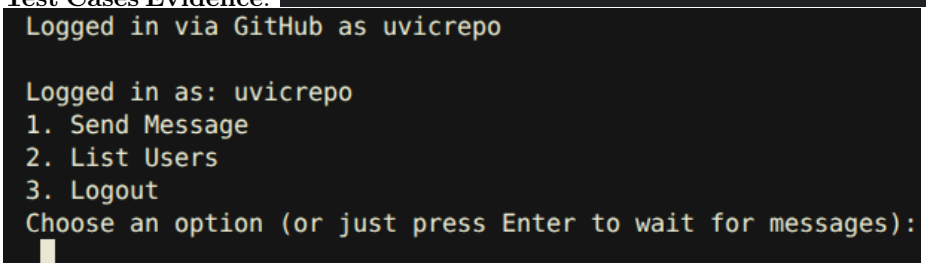
2.2.3 Challenges and Solutions The main design question was where account creation's password hashing was supposed to happen, client side or server side, since the assignment wording used the method name create_account() which exists on both sides. Looking at how authenticate() only exists on the server, and how the stored users.json file is only ever touched by server side code, made it clear that all hashing and salting had to be server side, with the client only ever sending a plaintext password over the (already unencrypted) socket.

While wiring this task in, I hit a bug where account creation appeared to work but then crashed on login, caused by the client hashing the password locally before sending it, so the server ended up hashing an already hashed value with its own separate salt. Fixing that meant having the client send the real password and letting the server handle hashing and salting entirely, consistent with how login already worked.

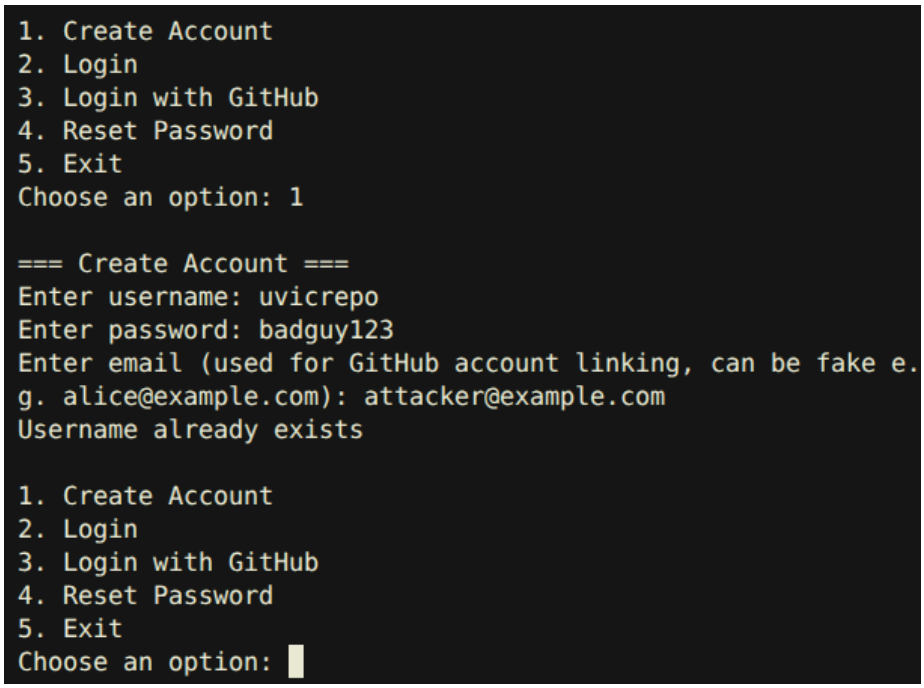
2.2.4 Testing and Validation Tested by creating a local account with a fake email, then logging in through GitHub with an account whose real email did not match, confirming a new linked account was created under the GitHub username. Also tested the case where a local account's email matched the GitHub account's primary email, confirming login resolved to the existing local account rather than creating a new one.



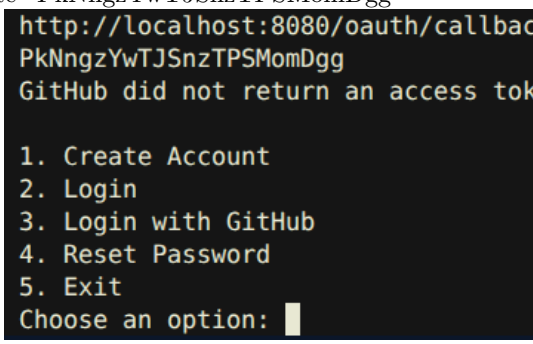
Test Cases Evidence:



Username conflict handling:



CSRF state rejection: original - code=2aa49b9cf7d67872a85e&state=PkNngzYwTJSnzTPSMomDgg



altered - code=2legit2quit&state=PkNngzYwTJSnzTPSMomDgg

2.3 Task 6: Zero Trust Implementation

2.3.1 Objective Apply Zero Trust ideas on top of the existing authentication: a basic challenge-response mechanism, role based restrictions on sensitive commands, session expiration, and logging with basic alerting.

2.3.2 Implementation Details Each account now has a second secret, challenge_key, separate from its TOTP secret. This separation exists because the console client itself needs to be able to compute an HMAC over an arbitrary server sent challenge on demand, which a phone based authenticator app cannot

do since it only implements the fixed TOTP algorithm. The TOTP secret is deliberately never given back to the client program after account setup, while the challenge key is written to a local file that the client reads automatically when a sensitive action requires re-authentication. Challenges are single use, since the server deletes a pending challenge as soon as it is checked, whether or not the response was correct, which prevents a captured response from being replayed later.

Roles are stored per user as either user or admin. Local accounts always start as user, and the only way to obtain admin is by logging in through GitHub, which keeps the admin provisioning path simple rather than adding a separate registration flow just for testing. LIST_USERS and a new VIEW_LOGS command are both restricted to admin accounts, and VIEW_LOGS also requires a successful challenge-response before it returns anything, so viewing logs needs both the right role and a fresh proof of possession of the challenge key

Sessions expire after a fixed number of commands rather than after a period of inactivity, since the server's main loop already blocks on reading the next message, and counting commands needed no new timers or socket level changes, while a true inactivity timeout would have required either a socket timeout or a background thread watching the clock independently of the main loop.

A TOTP check was added on the server side of RESET_PASSWORD, requiring a valid current code for the account being reset before the reset proceeds. Wiring the client side to actually prompt for and send that code was not carried out, since a fully working self-service reset flow was not a stated project requirement, only the restriction that only an admin can reset someone else's password.

Every command is now logged to a local file with a timestamp, the acting username, the command name, and the outcome. A simple in-memory counter tracks consecutive failed logins per username and prints a console warning after three in a row, resetting back to zero on the next successful login.

Key Components: - Component 1: Separate challenge-response key per account, stored locally by the client and never reused for TOTP - Component 2: Single use random challenges verified with HMAC-SHA256 and constant time comparison - Component 3: Role based restrictions on LIST_USERS and VIEW_LOGS - Component 4: Action count based session expiration - Component 5: Action logging to a file plus a console alert after three consecutive failed logins

Code Snippet (Key Implementation):

```
    def generate_challenge(self, conn):
        challenge = secrets.token_hex(16)
        self.pending_challenges[conn] = challenge
        return challenge
    # T6.A
    def verify_challenge_response(self, conn, username, response):
```

```
        challenge = self.pending_challenges.pop(conn, None)
        challenge_key = decrypt_totp_secret(self.users[username]['challenge_key'])
        return hmac.compare_digest(compute_mac(challenge_key.encode(), challenge), response)


    def _record_failed_login(self, username):
        self.failed_login_streak[username] = self.failed_login_streak.get(username, 0) + 1
        if self.failed_login_streak[username] >= 3:
            print(f"ALERT: {self.failed_login_streak[username]} consecutive failed logins fo

# T6.B - Admin-only command to list users and online status
                    #region LIST_USERS
                    elif command == 'LIST_USERS':
                        if not current_user:
                            response = {'status': 'error', 'message': 'Not logged in'}
                        elif self.users[current_user].get('role') != 'admin':
                            response = {'status': 'error', 'message': 'Admin only'}
                        else:
                            online_users = list(self.active_connections.keys())
                            all_users = list(self.users.keys())
                            response = {'status': 'success', 'online_users': online_users,

                    # T6.C - Admin-only command to view security log
                    #region VIEW_LOGS
                    elif command == 'VIEW_LOGS':
                        if not current_user:
                            response = {'status': 'error', 'message': 'Not logged in'}
                        elif self.users[current_user].get('role') != 'admin':
                            response = {'status': 'error', 'message': 'Admin only'}
                        else:
                            challenge_response = message.get('challenge_response')
                            if challenge_response is None:
                                response = {'status': 'challenge_required', 'challenge': sel
                            elif self.verify_challenge_response(conn, current_user, challeng
                                with open(LOG_FILE) as f:
                                    response = {'status': 'success', 'logs': ''.join(f.readl
                            else:
                                response = {'status': 'error', 'message': 'Challenge verific
```

2.3.3 Challenges and Solutions The biggest architectural problem in this task was discovering that any command sent to the server while already logged in was timing out. This turned out to be a pre-existing issue in the client's message handling thread, which only ever processed incoming chat messages and silently dropped anything else, including responses to commands like LIST_USERS or

the new VIEW_LOGS. Since account creation and login both happen before the listening thread starts, this bug had never been triggered by any testing up to that point. It was fixed by routing anything that is not a chat message into the same response queue that command sending waits on.

Deciding how the challenge-response mechanism should be used took a few iterations. Gating it on every login as a third factor alongside password and TOTP was one option, but using it instead as the re-authentication step for sensitive actions gave it a clearer, standalone purpose and kept it distinct from TOTP for comparison purposes.

2.3.4 Testing and Validation The challenge-response mechanism was tested directly: a correct response was accepted, an incorrect one was rejected, and replaying a response after its challenge had already been consumed was also correctly rejected. Role restrictions were tested by attempting LIST_USERS and VIEW_LOGS from both a local user account and a GitHub admin account, confirming the local account was denied both while the admin account succeeded. The failed login alert was tested by entering the wrong password three times in a row and confirming the warning appeared on the server console, then confirming a successful login reset the counter.

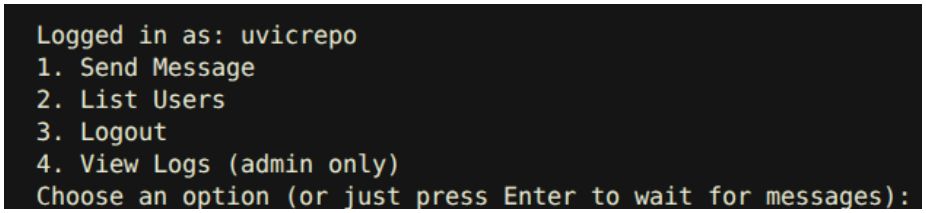
Test Cases Evidence: Establish admin account via github login:



Login with Github:



Logs in with Github username:



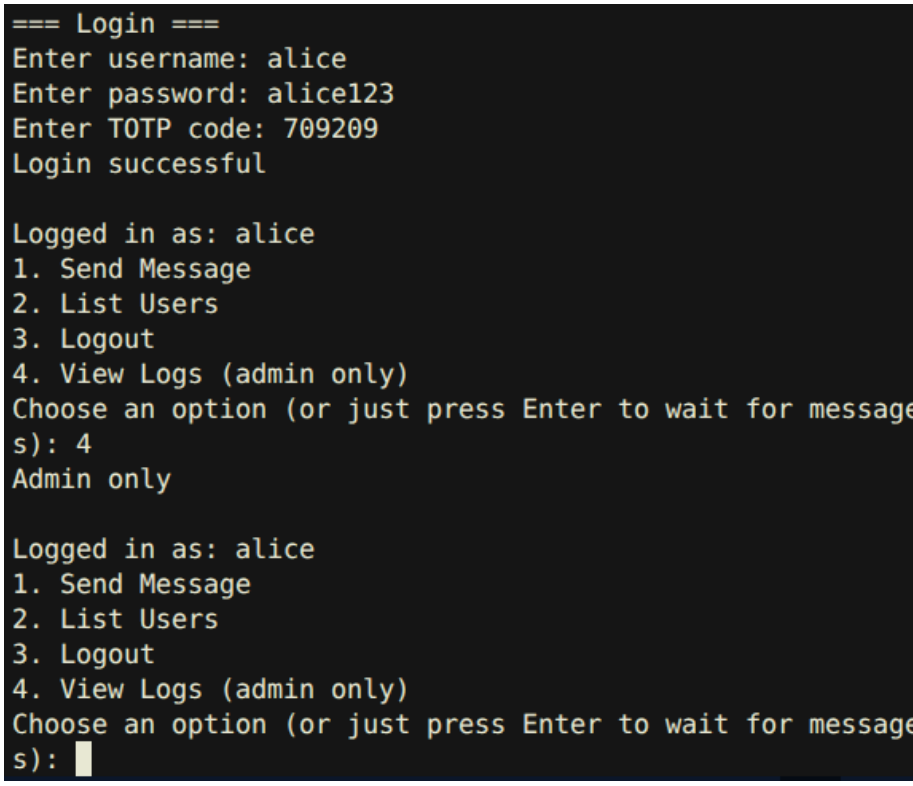
users.json show account set to admin:

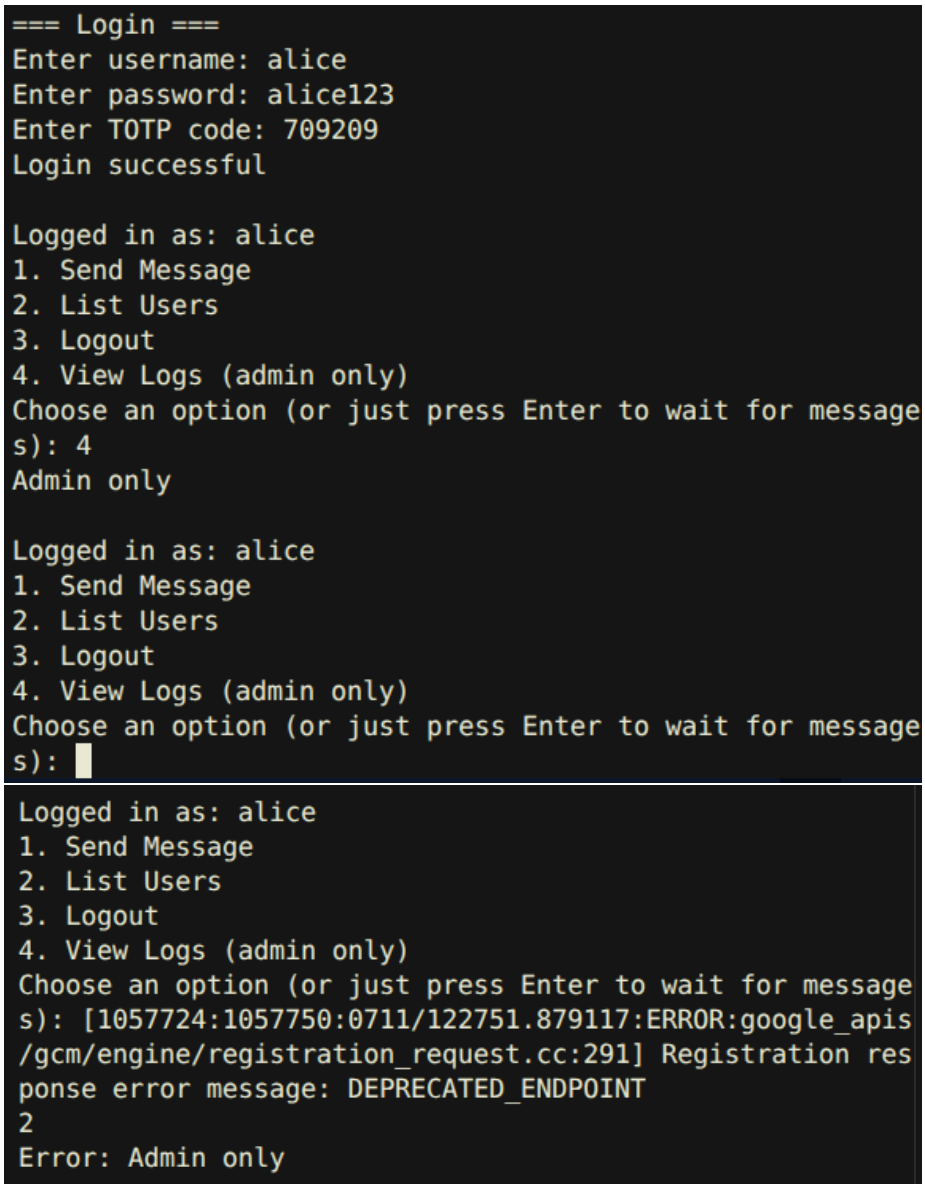


Logs are accessable with admin account:

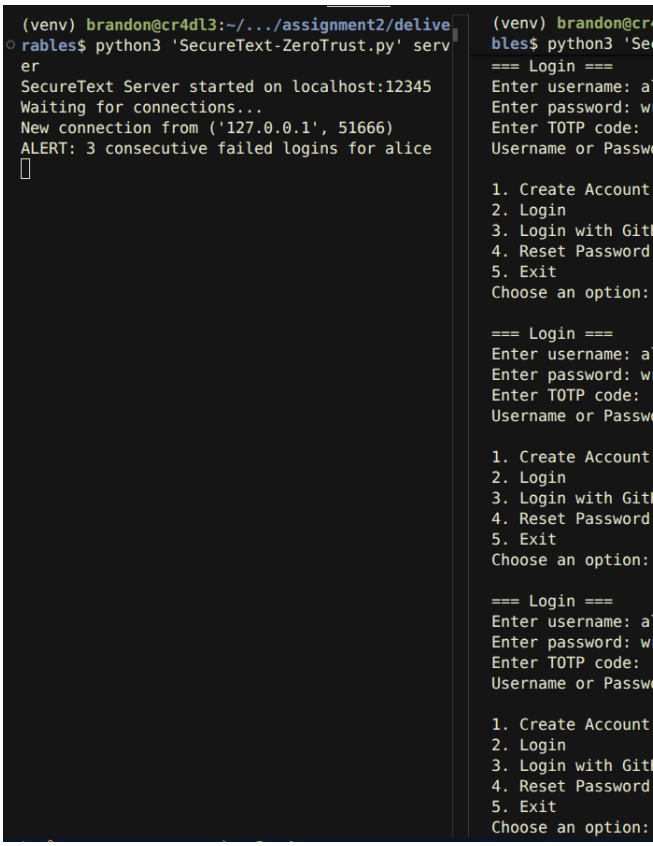


Not with a regular user account:

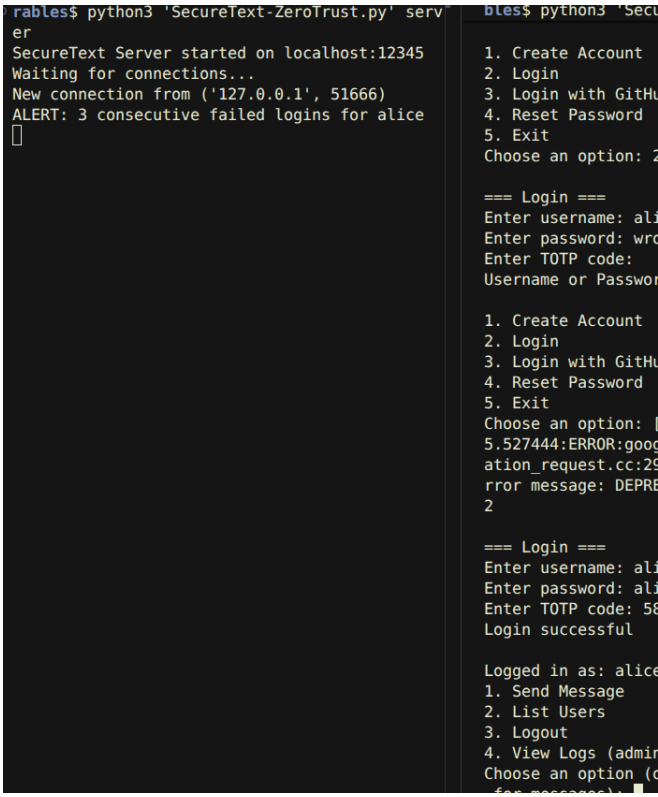



=== Login ===
Enter username: alice
Enter password: alice123
Enter TOTP code: 709209
Login successful
Logged in as: alice
1. Send Message
2. List Users
3. Logout
4. View Logs (admin only)
Choose an option (or just press Enter to wait for messages): 4
Admin only
Logged in as: alice
1. Send Message
2. List Users
3. Logout
4. View Logs (admin only)
Choose an option (or just press Enter to wait for messages):
Logged in as: alice
1. Send Message
2. List Users
3. Logout
4. View Logs (admin only)
Choose an option (or just press Enter to wait for messages): [1057724:1057750:0711/122751.879117:ERROR:google_apis/gcm/engine/registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
2
Error: Admin only

Alerts generate for 3 consecutive failed authentications:



Streak resets after 3 consecutive failed authentications:



---

3. Security Analysis

3.1 Vulnerability Assessment

Identified Vulnerabilities (Assignment 2): | Vulnerability | Severity | Impact | Location(function/action) | Mitigation | |————————|————-|————|————-|————| | Password-only authentication | High | A correct password alone was sufficient to log in, so a leaked or guessed password gave full account access | Login flow / authenticate() | TOTP added as a required second factor at login; GitHub OAuth added as an alternative login path not tied to a locally stored password | | User enumeration | Medium | Different error messages or “List Users” for valid versus invalid usernames let an attacker discover which accounts exist | Login and password reset flows | Switched to amigious error messaging and admin only List Users function | | Non-authorised password resets | High | Anyone could reset any account’s password by supplying only a username, with no proof of ownership | reset_password() / RESET_PASSWORD command | admin can only reset passwords |

3.2 Security Improvements

Before vs. After Analysis: - **Authentication**: Login now requires a password plus a TOTP code, and a GitHub login path was added as an alternative. - **Authorization**: Roles were introduced for the first time, with user and admin accounts, and two commands are now restricted to admins. Previously every logged in account had identical access to every command.
- **Data Protection**: TOTP secrets and challenge-response keys are encrypted at rest with AES-256-GCM rather than stored in plaintext. - **Communication Security**: No change.

3.3 Threat Model

Passive Network Attacker: Can still read every credential and message sent, since there is no transport encryption. TOTP codes are single use over a short window, so intercepting one is of limited value compared to intercepting a password.

Active Network Attacker: Could still modify or inject traffic on the socket itself, but cannot forge a valid OAuth callback due to the state check, and cannot forge a valid TOTP or challenge-response answer without the underlying secret.

Malicious Server Operator: Has access to users.json and could read hashed passwords, encrypted TOTP secrets, and encrypted challenge keys, but would need the server's own AES key.

Compromised Client: An attacker with access to the user's machine could read the local challenge-response key file directly, and would have access to an already logged in session, but would not automatically gain the TOTP secret, since that only ever lived in the authenticator app and not on the client machine.

Use the following security properties and threat actors in your threat modeling. You can add extra if needed.

Threat Actors: 1. **Passive Network Attacker**: Can intercept but not modify traffic 2. **Active Network Attacker**: Can intercept and modify traffic 3. **Malicious Server Operator**: Has access to server and database 4. **Compromised Client**: Attacker has access to user's device

Security Properties Achieved: - [x] Confidentiality - [x] Integrity
- [x] Authentication - [x] Authorization - [ ] Non-repudiation - [ ] Perfect Forward Secrecy - [ ] Privacy

---

4. Attack Demonstrations

4.1 Attack 1: Password Only Authentication Bypass Attempt

4.1.1 Objective Show that knowing a user's correct password alone is not enough to log in once TOTP is enabled.

4.1.2 Attack Setup Tools Used: - Tool 1: SecureText client: used to attempt login with a correct password and an incorrect or missing TOTP code

4.1.3 Attack Execution

1. An account is created normally, generating a TOTP secret tied to that account.
2. Login is attempted using the correct username and password, but with a TOTP code that does not match the account's secret.
3. The server checks the password first, then checks the TOTP code separately, and rejects the login if either check fails.

4.1.4 Results and Evidence Evidence: See Task 4 Evidence section.

4.1.5 Mitigation This is the mitigation itself rather than something needing a further fix. TOTP was added specifically so that password compromise alone is not sufficient for account takeover.

4.2 Attack 2: OAuth State Tampering

4.2.1 Objective Show that a tampered or mismatched state value on the OAuth callback is rejected rather than allowing the login to continue. #### 4.2.2 Attack Setup
SecureText client and server: the pasted back redirect URL was manually edited before sending it to the server #### 4.2.3 Attack Execution 1. A normal GitHub login is started, and the server issues a random state value tied to that connection. 2. Before pasting the redirect URL back into the console, the state value in the URL is changed to a different string. 3. The client forwards this modified URL to the server as normal.

4.2.4 Results and Evidence See Task 6 Evidence section. #### 4.2.5 Mitigation This is already the mitigation for CSRF style attacks against the OAuth flow. The random per-connection state value being checked before anything else happens is what stops a forged or replayed callback from completing a login.

---

5. Performance Evaluation

This assignment focused on adding authentication features rather than benchmarking, so performance testing here is limited to basic timing observations rather than a full study.

---

6. Lessons Learned

6.1 Technical Insights

1. **Insight 1**: Key separation matters even in a small app: using one key for two different cryptographic purposes, such as message HMAC and TOTP encryption, would remove the isolation that lets one key be rotated or compromised without affecting the other.
2. **Insight 2**: Encryption and hashing are not interchangeable for secret storage: TOTP and challenge-response secrets need to be recovered later to compute an expected value, so they had to be encrypted, not hashed, unlike passwords which only ever need a match comparison.
3. **Insight 3**: A shared secret and a rotating code from it are two different things: TOTP only needed a secret generated once, at account creation, since the code itself is derived fresh every 30 seconds from that fixed secret and the current time. This was the source of my original confusion about why the assignment asked for secret generation "at account creation" for something described as time based.

6.2 Security Principles

Applied Principles: - **Defense in Depth**: Login requires both a password and a TOTP code, and sensitive actions require a further challenge-response step on top of an already authenticated session. - **Least Privilege**: Regular accounts cannot list all users or view logs, only admin accounts can, and nobody can reset a password they do not control. - **Fail Secure**: A session that hits its action limit is forced back to a logged out state rather than continuing to trust the existing session, and a missing or incorrect challenge-response answer results in denial rather than a default allow. - **Economy of Mechanism**: Session expiration was implemented as a simple action counter rather than a timer or background thread, and admin provisioning reuses the existing GitHub OAuth login rather than building a separate registration path.

---

7. Conclusion

7.1 Summary of Achievements

TOTP based two factor authentication, GitHub OAuth login, and a set of Zero Trust features including challenge-response re-authentication, role based access control, session expiration, and logging with basic alerting were all implemented and tested successfully on top of the Assignment 1 codebase. Several pre-existing bugs, including an inverted password check, a client side password double hashing issue, and a dropped-response bug in the client's message handling, were also identified and fixed along the way.

7.2 Security and Privacy Posture Assessment

Remaining Vulnerabilities: - Vulnerability 1: No transport encryption. All traffic, including passwords, TOTP codes, and pasted OAuth URLs, is sent in plaintext over the socket, so a passive attacker on the same network segment can read everything. - Vulnerability 2: An OAuth-only account has no password or TOTP secret, so if that account ever attempted a local login or password reset it would fail in an unhandled way rather than with a clean error message. This was left as a known limitation given the time available.

Suggest an Attack: In two lines mention a possible existing attack to your current version in abstract

7.3 Future Improvements

A passive attacker capturing loopback or LAN traffic could read a TOTP code as it is entered and reuse it within its 30 second window before the legitimate login completes, since nothing about the transport itself protects that value in flight.

1. **Improvement 1**: Add transport encryption, such as TLS, so that credentials and TOTP codes are not visible to anyone able to observe network traffic.
2. **Improvement 2**: Replace the action count based session timeout with a true inactivity timeout, and add PKCE to the OAuth flow, both of which were left out or simplified due to time constraints rather than for a technical reason.

---

8. References

- RFC 6238, TOTP: Time-Based One-Time Password Algorithm, https://www.rfc-editor.org/rfc/rfc6238 pyotp documentation, https://pyauth.github.io/pyotp/
- PyCryptodome AES documentation, https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html
- qrcode Python library documentation
- Google Authenticator Key Uri Format, https://github.com/google/google-authenticator/wiki/Key-Uri-Format
- GitHub OAuth Apps documentation, https://github.com/settings/developers and GitHub's OAuth web application flow documentation
- NIST SP 800-57, Recommendation for Key Management, referenced for key separation reasoning

---

Submission Checklist

Before submitting, ensure you have:

- ☐ **Complete Report**: All sections filled out with sufficient detail
- ☐ **Evidence**: Screenshots, logs, and demonstrations included
- ☐ **Code**: Well-named(based on task and whether it is an attack or a fix) and well-commented and organized in your GitHub repository deliverable directory of the corresponding assignment
- ☐ **Tests**: Security and functionality tests implemented after fix
- ☐ **GitHub Link**: Repository link included in report and Brightspace submission
- ☐ **Academic Integrity**: All sources properly cited, work is your own
- ☐ **AI Disclosure**: `GENAI.md` included in deliverables, disclosing any AI use

---

Submission Instructions: 1. Save this report as PDF: `[StudentID]_Assignment[X]_Report.pdf` 2. Submit PDF to Brightspace 3. Include your private GitHub repository link in the Brightspace submission comments 4. Ensure your repository is private until after course completion otherwise you'll get zero grade

Final Notes: - Disclose AI use in a `GENAI.md` file in your deliverables (prompts used, which files AI touched). See the Academic Integrity section of the root README. - Be ready to explain any code you submit. If you can't explain it, don't submit it. - Be honest about limitations and challenges - Focus on demonstrating understanding, not just working code - Quality over quantity: depth of analysis is more important than length - Proofread for clarity and technical accuracy

Good luck!